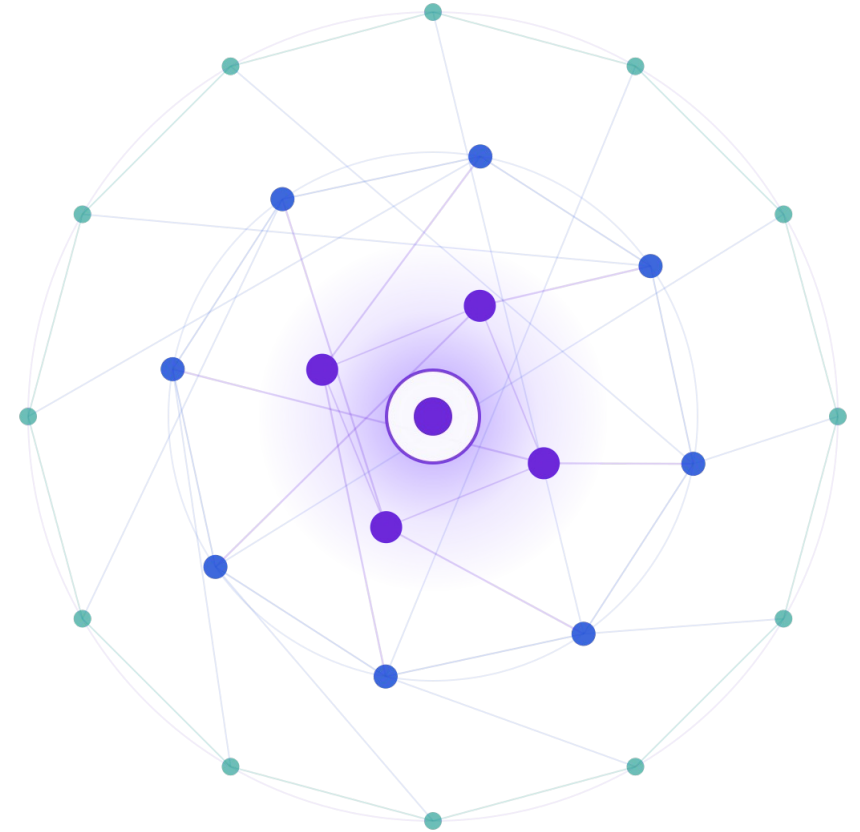


Generative AI Hackathon

3rd Edition 2026 – 27

- Theme ID - 4 (Streaming Live RAG)
- Team Name - Pokermans
- College Name - VITV
- Member Name & Email 1-
- Member Name & Email 2-
- Member Name & Email 3-
- Member Name & Email 4-
- Submission Github link - (add repo link)

DuplexRAG - retrieve while the user is still speaking



Theme

Theme 04 · Streaming Live RAG

In a full-duplex voice conversation people speak one natural request, not a tidy query.

- **Hidden intents** - one utterance often carries several questions.
- **Latency** - waiting for the end of speech before searching leaves dead air.
- **Late details** - "actually it's fifty people" must refine the answer, not restart it.
- **Not every turn needs search** - "repeat that in two bullets" is formatting.
- **Trust** - every claim must be grounded in the provided corpus, and gaps must be said out loud.

Goal: a cheap, CPU-only engine that retrieves during speech, answers every hidden intent with citations, and keeps the answer alive across the conversation.

One utterance, three hidden questions

0.0 s "I need to plan a customer workshop in..."

0.8 s "...Pune for 30 people, and I need..."

1.6 s "...the cancellation policy and the catering options."

venue capacity
for 30

cancellation
terms

catering options

then: "Actually it's going to be fifty."

→ **refine the capacity claim, keep the rest (v1 → v2)**

then: "Can you repeat that in two bullets?"

→ **no retrieval, same citations**

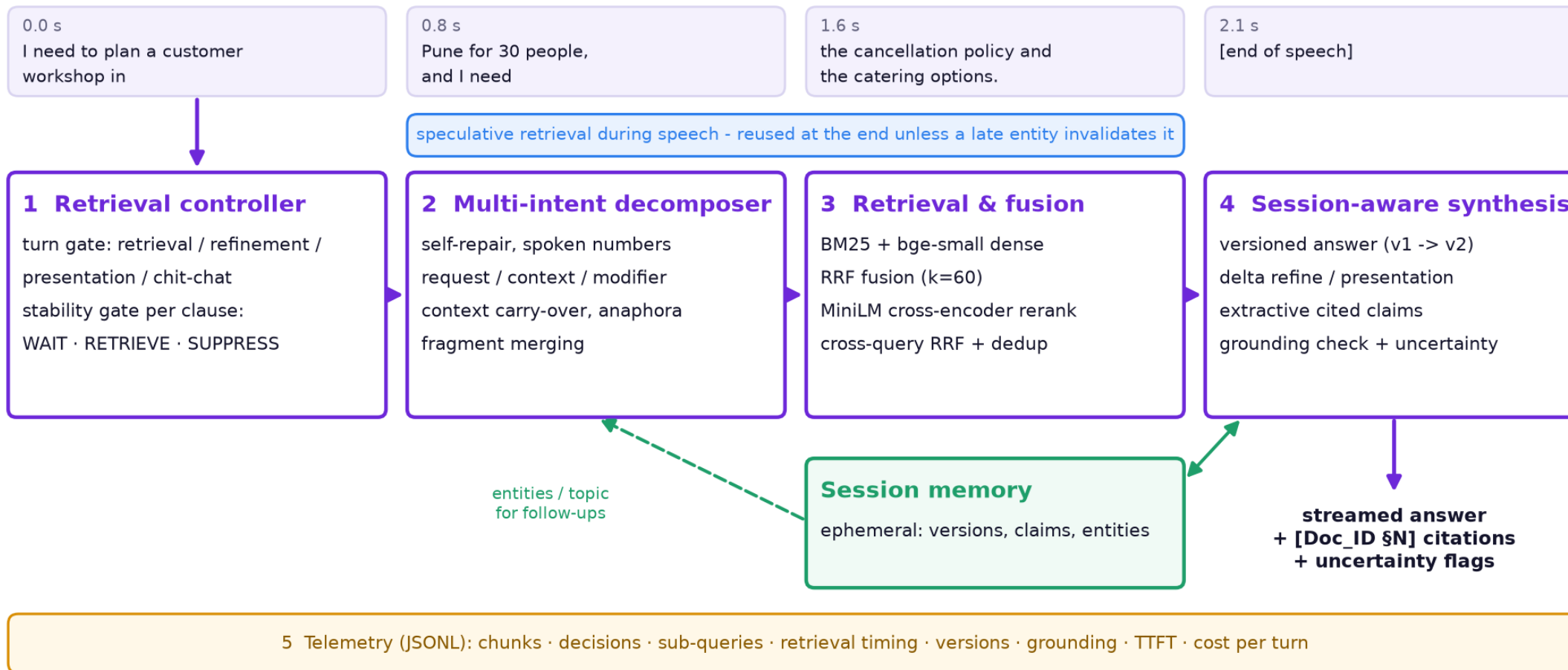
Existing Solutions & Gaps

Approach	How it works	Gap for full-duplex voice
Turn-based RAG	waits for end of speech, one query for the whole utterance	dead air while searching; compound requests get one blurred query
LLM query rewriting / multi-query (RAG-Fusion, HyDE)	an LLM rewrites or splits the query after the user stops	adds an LLM call to every turn: more latency and cost, and it can invent queries
Agentic RAG (plan → search → read loops)	multi-step tool use by an LLM agent	heavy orchestration and unpredictable latency - the brief asks for simple and cheap
Chat-history RAG	concatenates history into the next query	late details restart the search; context drifts; old citations are lost
Intent-based voice assistants	classify intent → fixed skill	no grounding in documents, no citations, no answer for long-tail policy questions

The gap: nothing retrieves during speech, refines the answer in place and proves every claim - on a CPU budget, without an LLM in the loop.

Our Solutions & Architecture Diagram

Incoming ASR stream (one natural request, several hidden questions)



Event-driven: on_chunk() per ASR partial, end_turn() at end of speech; retrieval jobs run in the background while the user keeps talking.

Demo & Product Walkthrough

[ui.png missing]

1. Stream replay a scenario, type, or speak (Chrome mic) - chunks arrive every ~1 s

2. Watch the controller WAIT / RETRIEVE / SUPPRESS per chunk; provisional and multi-intent searches fire mid-sentence

3. Grounded answer one line per hidden intent, every claim cited; click a citation to see the source section

4. Late detail "actually fifty people" → v2 with a diff: added / retained / retired claims, delta search only

5. Format request "two bullets" → no retrieval, same citations; the trace and cost are downloadable

Demo video:

[submission/DuplexRAG_demo.mp4](#)

Tools and tech stack used

Models (CPU, ONNX)

- BAAI bge-small-en-v1.5 - 33M bi-encoder
- ms-marco-MiniLM-L-6-v2 - 22M cross-encoder
- fastembed + ONNX Runtime, 4 threads
- no GPU, no API keys, runs offline

Retrieval

- Okapi BM25 (NumPy, in-house)
- dense cosine search
- reciprocal-rank fusion (k=60)
- cross-encoder rerank + sentence scoring

Language layer

- rule-based spoken-language decomposer
- turn gate: rules + NumPy softmax classifier
- self-repair, spoken numbers, anaphora
- optional OpenAI-compatible LLM (verified)

Serving & UI

- FastAPI + WebSocket (uvicorn)
- vanilla JS live UI, Web Speech API mic
- CLI: index / serve / replay / ask / bench
- JSONL telemetry + JSON Schema

Engineering

- Python 3.12, uv lockfile
- Docker Compose (one command)
- pytest + smoke test (G1)
- git, periodic commits

Evaluation

- discrete-event streaming replay
- 4 benchmark splits, 3 written independently
- baseline + 7 ablations, gates G1-G6
- matplotlib figures, Playwright demo capture

Impact & Use case

Voice assistants

answer 'how do I...' questions from manuals, policies and settings docs while the user is still talking - with a source for every claim

n/a

median head start:
retrieval begins before the
user stops talking

n/a

median time to first token
after speech

Contact-centre agent assist

surface the right policy clauses during a live call; refine as the caller adds details; no dead air

n/a

same stack without
speculation

n/a

compute cost per turn
(CPU only)

Enterprise helpdesk (HR / IT / travel)

compound, messy questions answered in one turn with explicit 'not in the policy' flags

n/a

claims supported by their
citation

n/a

fabricated citation IDs

Hands-free field work

technicians and drivers ask by voice; CPU-only, offline-capable, cheap to run

held-out test split, laptop CPU (Ryzen 5 4600H)

Innovation highlights, results and limitations

Held-out test (30 sessions, 74 turns)	DuplexRAG	Baseline	Target
G2 retrieval starts before end of speech	n/a	n/a	$\geq 80\%$
G2 false triggers on no-retrieval turns	n/a	n/a	low
G3 compound turns: ≥ 2 intents isolated	n/a	n/a	$\geq 70\%$
G4 claims supported by citation	n/a	n/a	$\geq 85\%$
G4 fabricated citation IDs	n/a	n/a	0
G5 refinements: versioned, delta-only	n/a	n/a	pass
G6 telemetry trace coverage	n/a	n/a	100%
Retrieval hit@3 (gold section)	n/a	n/a	
Key facts present in answer	n/a	n/a	
TTFT p50 after end of speech	n/a	n/a	

Baseline = turn-based RAG: waits for end of speech, one dense query, no rerank, no session memory. G1: one-command Docker build + automated replay; smoke test passes from a clean checkout.

Innovation highlights

- speculative per-clause retrieval with entity-aware reuse
- refine-in-place answer versions with claim diffs
- grounded by construction; claim-level verifier
- entity x aspect gaps said out loud
- no LLM needed: about \$0.0001 per turn

Limitations

- synthetic corpus (official corpus unavailable)
- extractive wording is less fluent than an LLM
- conservative 'not found': some unanswerables still get a related, cited answer
- speech simulated from transcripts (live mic in Chrome)

What's next

1

Plug in the official corpus + real ASR partials

streaming Whisper / on-device ASR; tune the stability gate on real partial-hypothesis revisions

2

Learned stability controller

train the retrieve / wait policy from ASR confidence, prosody and pauses

3

Small verified on-device LLM

fluent phrasing through the existing claim verifier; stream into TTS

4

Multilingual and code-mixed speech

Hindi-English, Korean; multilingual bi-encoder and cross-encoder

5

NPU / quantised models

int8 cross-encoder and cached sentence scores for sub-100 ms p95 on phones

6

Barge-in aware answers (with Theme 5)

stop, refine and resume the spoken answer when the user interrupts

Brownie points slide(differentiation)

Retrieves while you talk

searches fire mid-sentence and are reused

Zero fabricated citations

n/a of claims verified against the cited section; IDs cannot be invented

CPU-only, about \$0.0001 per turn

two small ONNX models, no GPU, no API keys, works offline in Docker

Refine, don't restart

versioned answers with added / retained / retired claims; only the delta is searched

Says what it doesn't know

"catering for Hinjewadi Tech Park could not be verified" - entity x aspect coverage checks

Measured honestly

discrete-event replay; held-out sets written independently after freezing; the earlier held-out run is reported too

Checklist- Updated on Public GitHub

- Working prototype code — public or shared GitHub repo (Y): (add public repo link)
- README with reproducible setup instructions (Y): README.md, Docker Compose, smoke test
- Demo video, max 5 minutes (Y): submission/DuplexRAG_demo.mp4 in the repo
- Presentation file (PPT or PDF) (Y): submission/VITV_Pokermans_Submission.pptx

Thank you

Organised by the Language AI Team and the PRISM Team, Samsung R&D Institute India